

Professional Scrum Developer Glossary

★4.4 from 47 ratings

Save 

This glossary represents an overview of terms specific to software development teams using Scrum and agile software development techniques.

To learn more about the Scrum framework, we highly recommend that you reference the [Scrum Guide™](#) and the [Scrum Glossary](#).

A

A/B Testing: extends the idea of hypothesis driven development by evaluating two or more different implementations to find out which one works best. Usually this is done by having different implementations and then route a part of our users to each of them. This allows to measure which implementation better supports the expected user behavior. A/B Testing is often combined with Feature Flags and Application Telemetry.

Acceptance Test-Driven Development (ATDD): test-first software development practice in which acceptance criteria for new functionality are created as automated tests. The failing tests are constructed to pass as development proceeds and acceptance criteria are met.

Application Lifecycle Management(ALM): holistic view on the management of software applications and systems, accounting for all stages of the existence of a software product.

Application Telemetry: Understanding how a product is used is a key factor for taking better decisions on where to invest. Application Telemetry can provide some insights to increase this understanding by showing usage statistics, performance parameters, user workflows and other relevant information.

B

Behavior-Driven Development (BDD): agile software development practice adding to TDD the description of the desired functional behavior of the new functionality.

Blameless Postmortem: is to understand systemic factors that lead to an outage and identify learnings and actions that can help to prevent this kind of failure from recurring. This practice is based on the idea that in hindsight we usually know how the outage could have been prevented. But the past cannot be changed and therefore it is useless to discuss who should have done what, aka as blaming. But it is about shaping the future by learning from what just happened. What can we learn and how can we improve our process to make it more resilient?

Blue-Green Deployment: is a practice that helps reducing down-times while upgrading the system to a new version. It has other positive effects like fast rollbacks in case of emergency. It uses two identical environments. One environment (called blue to differentiate it from the other identical one) is handling all requests and executing all production operations. The other environment (green) can handle software updates and configuration changes without impacting production. Even tests can be executed on the green environment without risk. Once the green environment is ready, all requests are switched over to this one and it becomes the new blue environment. The previous blue environment at the same time becomes the green one and can be used for the next update.

Branching: creating a logical or physical copy of code within a version control system so that this copy might be changed in isolation.

C

Clean Code: software code that is expressed well, formatted correctly, and organized for later coders to understand. Clarity is preferred over cleverness.

Code Coverage: a measurement indicating the amount of product code that is exercised by tests.

Cohesion and Coupling: coupling refers to the interdependencies between modules, while cohesion describes how related the functions within a single module are.

Collective Code Ownership: a software development principle popularized by Extreme Programming holding that all contributors to a given codebase are jointly responsible for the code in its entirety.

Continuous Delivery: a software delivery practice similar to Continuous Deployment except a human action is required to promote changes into a subsequent environment along the pipeline.

Continuous Deployment: a software delivery practice in which the release process is fully automated in order to have changes promoted to the production environment with no human intervention.

Continuous Integration (CI): agile software development practice popularized by Extreme Programming in which newly checked-in code is built, integrated and tested frequently, generally multiple times a day.

Continuous Testing: Traditional approaches of quality assurance are often based on getting all implementation finished before verifying the latest build of the product before delivering it to production. In contrast continuous testing is a practice of integrating testing as a fundamental and ongoing part of development. It helps to identify and fix issues much earlier and so lowers risk drastically. Continuous testing is especially powerful if combined with practices like test automation, clean code and others which help to reduce regression testing efforts.

Cycle Time: is the time between working on an item that has been started and the item is finished (usually delivered to real end-users). Cycle Time defines how fast work can flow through a system and minimizing Cycle Time helps not only to make the system more efficient but also to increase predictability and the ability to quickly respond to changes or new insights.

Cyclomatic Complexity: a measure of code complexity based on the number of independent logical branches through a codebase. Cyclomatic complexity is expressed as a simple integer.

Cross-functional: characteristic of a team holding that all the skills required to successfully produce a releasable Increment in a Sprint are available within the team, where releasable refers to making the software available in production.

D

Definition of Done: a shared understanding of the expectations that software must live up to in order to be releasable into production, with a purpose of providing transparency over the software created.

Developer: any member of a Scrum Team, that is committed to creating any aspect of a usable Increment each Sprint regardless of technical, functional or other specialty.

DevOps: an organizational concept serving to bridge the gap between development and operations, in terms of skills, mind-set, practices and silo-mentality. The underlying idea is that developers are aware of, and in daily work consider implications on operations, and vice versa.

Dev & Ops collaboration: is at the core of the DevOps movement. Instead of separating development and operations, collaboration is key. Instead of developing something and then make running this in production someone else's problem, we try to achieve end-to-end responsibility. This not only helps with smoothening the delivery process, but it also closes the feedback loop. A closer collaboration strengthens learning how we could support robust operation already in the design and implementation operations.

DRY (don't repeat yourself): software development principle to avoid repetition of the same information in one system, preventing the same code from being produced multiple times on a codebase.

E

Engineering Standards: a shared set of development and technology standards that the Developers apply to create releasable Increments of software, and against which those Developers can inspect and adapt.

Error Culture: How mistakes are handled has an important impact on an organizations ability to innovate. If people feel that errors are something negative and try to avoid them, they might be much less likely to take a risk and try something new. Instead encouragement for experimentation and learning is important while at the same time considering how to tame risks and decrease the impact of failures.

Extreme Programming (XP): agile software development framework with an extreme focus on programming and taking engineering practices to an extreme in order to create and release high quality code. Highly complementary to the Scrum framework.

F

Feature Flags/Feature Toggle: software development practice that allows dynamically turning (parts of) functionality on and off without impacting the overall accessibility of the system by its users.

H

Hypothesis Driven Development: The basic idea behind Hypothesis Driven Development is that in a complex environment we do not know where to invest in order to achieve the highest possible value: we formulate hypotheses about that. Once we accept uncertainty and agree that the assumptions our plans are based on can be wrong, it makes sense to change our approach to the development of new features. Validating our assumptions gets high priority and finding small and fast experiments to get more insights becomes an important part of work.

I

Increment: a fully functional piece of working software, living up to the Definition of Done, that adds to previously created Increments, where the sum of all Increments - as a whole - form a product. The moment a Product Backlog item meets the Definition of Done, an Increment is born.

Infrastructure as Code: Instead of setting up and configuring infrastructure and environments, this process can be automated by scripts and parameter files. While this approach is not faster for the initial setup of the infrastructure (it might even be slower), it provides a lot of advantages. The scripts and configuration files can be stored in version control together with the source code of the software. This allows to create exactly the matching environment for a given version of the software. Changes to the environment are documented in version control as they are no longer executed on the environment directly but by changing and executing a script. And new instances of an exact copy of the production environment can easily be created, for example for testing purpose.

M

Mean Time to Detect (MTTD): is the time it takes to identify that there is a problem. The MTTD should not be determined by the first call of an angry customer at the hotline. Monitoring tools can help to reduce it.

Mean Time to Recover (MTTR): is the average time it takes from when a problem occurs until the problem is fixed and the system is back to normal operations. There are various techniques helping with MTTR as defensive programming and self-healing systems that can switch to an emergency mode to keep the basic functionality of the system up and running. As MTTD is usually a significant portion of MTTR, reducing MTTD will also help with MTTR.

Minimum Viable Product: One practice to achieve Hypothesis Driven Development is the Minimum Viable Product (MVP). The MVP is the smallest implementation of our product or a feature which allows to learn more about how users will react to it or technical behavior like performance

Monitoring: Obviously, it is helpful to know the current state of your system. While most systems support logging to analyze what happened during an outage or a problem, monitoring is used to check the current state continuously. Once one or more parameters get out of a healthy range, alarms can be triggered to initiate some actions

N

Non-functional Requirement: Non-functional requirements define criteria to evaluate a product's operation, not its specific behaviors. They describe conditions for the product's effectiveness or qualities it must possess. Common types include availability, reliability, regulatory compliance, maintainability, and safety.

P

Pair Programming: agile software development practice popularized by Extreme Programming in which two team members jointly create new functionality.

R

Refactoring: agile software development practice popularized by Extreme Programming in which code is adjusted within the codebase without impacting the external, functional behavior of that code.

Release-Pipelines: Automating the steps from code commit into version control to delivery in production help increasing speed and reliability of this process. This practice is often referred as Release-Pipelines as this pictures the ideal of a steady stream of changes delivered.

S

Scout Rule: the practice of always leaving the codebase in a little better state than it was found before modifications. A means to progress towards Clean Code.

Scrum: a framework to support teams in complex product development. Scrum consists of Scrum Teams and their associated accountabilities, events, artifacts, and rules, as defined in the [Scrum Guide™](#) [↗](#).

Scrum Board: a board to visualize information within the Scrum Team primarily, often used to manage Sprint Backlog. Scrum boards are a complimentary practice within Scrum to make information visible and thereby increase transparency.

Scrum Guide™: the definition of Scrum, written and provided by Ken Schwaber and Jeff Sutherland, co-creators of Scrum. The [Scrum Guide](#) [↗](#) consists of Scrum's accountabilities, events, artifacts, and the rules that bind them together.

Scrum Team: a self-managing team consisting of a Product Owner, Development Team and Scrum Master.

Self-Managing: Scrum Teams are cross-functional, meaning the members have all the skills necessary to create value each Sprint. They are also self-managing, meaning they internally decide who does what, when, and how.

Specification by Example: agile software development practice based on TDD and ATDD that calls for using realistic examples from past experience instead of untested or abstract statements in the description of the desired functional behavior.

T

Test-Automation: Automating tests is not only a way to increase the quality of a product, it also helps to reduce cycle time. If you see test execution as one form of feedback, automated tests help to get this feedback much earlier. This lowers the effort of fixing issues and allows to deliver releasable increments much faster.

Test-Driven Development (TDD): test-first software development practice in which test cases are defined and created first, and subsequently executable code is created to make those tests pass. The failing tests are constructed to pass as development proceeds and tests succeed.

Technical Debt: the typically unpredictable overhead of maintaining the product, often caused by less than ideal design decisions, contributing to the total cost of ownership. May exist unintentionally in the Increment or introduced purposefully to realize value earlier.

Testing in Production: What sounds like a very dangerous approach can help not only to reduce cycle time but also make your tests more reliable. It means to execute various tests right in the production environment. To reduce the risk of affecting production operations with untested and failing functionality, various technologies can be used to make the new functionality only available to the test process or a very small set of users while other users will use the previous functionality. The change will be rolled out to all users once the test was successful. This practice will help you to get rid of various test stages like QA, UAT, Staging etc. And as the tests run in the real production environment, you can eliminate the risk of using a test environment that behaves slightly different from your production environment.

U

User Story: agile software development practice from Extreme Programming to express requirements from an end user perspective, emphasizing verbal communication. In Scrum, it is often used to express functional items on the Product Backlog.

Unit Test: low-level technical test focusing on small parts of a software system that can be executed fast and in isolation. The definition and boundaries of a 'unit' generally depends on the context and is to be agreed upon by the Developers

V

Velocity: an optional but often used indication of the amount of Product Backlog turned into an Increment of product during a Sprint. It is tracked by the Developers for use within the Scrum Team.

Vertical Teams: In traditional organizations, there are many different teams and departments involved in the process from a customer providing feedback or suggesting an improvement to delivering the new version to customers. Sales & Marketing, Product Management, Development, Quality Assurance, Operations and more jump to mind. But each different department or team means there is a handoff. Handoffs tend to be slow and error-prone. In contrast a vertical team combines all the necessary competencies to handle the whole process end-to-end. In such scenarios teams typically are only responsible for small parts of the whole product. So instead of splitting the organization into horizontal layers (departments) this approach suggests slicing the product.

What did you think about this content?



4.4 from 47 ratings

About

[Who is Scrum.org](#)

[Latest News](#)

[Partners](#)

[Support Center](#)

Quick Links

[Class Schedule](#)

[Find a Trainer or Request a Private Class](#)

[Resources](#)

[The Scrum Guide](#)

Social media

